

The Code Book for Prompt Coding

The final codebook is designed as a sequence-oriented coding scheme because the dataset does not consist of isolated prompts, but of three-round prompt sequences per case. Accordingly, the unit of analysis is one case, defined as one person, one row, and one complete three-round sequence. Within each case, Round 1, Round 2, and Round 3 are coded separately. After the three rounds have been coded, the overall sequence is coded as an additional analytical unit.

For each round, coding always follows the same order. First, a function code is assigned. F1 denotes a generation prompt, that is, a prompt that describes an image from scratch. F2 denotes an edit prompt, that is, a prompt that changes an already existing image or result. F3 denotes a re-generation or complete reformulation, meaning that a later round again describes an image almost completely from scratch rather than modifying the existing one locally. F4 denotes a meta- or repair prompt, meaning that the prompt corrects the process rather than primarily the image itself, for example by indicating a mistake or instructing the system to return to a previous image. The core decision rule is: if the prompt clearly refers to an existing image, it is coded as F2; if it formulates a new full image description again, it is coded as F3.

In addition, the codebook applies a **prompt-engineering level** to all prompts, that is, to F1, F2, F3, and F4. This level operationalizes the degree of control present in the prompt and is assessed on the basis of four guiding questions: how precise the subject is, how clear form, composition, or perspective are, how well style, modifiers, and quality specifications are developed, and whether constraints or deliberate limitations are present. L1 denotes low prompt engineering: the prompt contains only the basic motif, with little control, little composition, hardly any style specification, and no clear constraints. L2 denotes medium prompt engineering: the motif is clearly controllable, and several relevant specifications are present, such as a clear motif description, a recognizable setting, and initial perspective or style control. L3 denotes high prompt engineering: the prompt strongly controls image effect, composition, and style, using precise spatial cues, consistent style, several detail modifiers, strong controllability, explicit relations, and deliberate constraints that leave little room for interpretation.

As an additional analytical dimension, the codebook also captures **vibe prompting**. This dimension is intended to identify prompts that shift from the specification of concrete visual elements toward the specification of an intended feeling, atmosphere, or subjective impression. A prompt is coded as containing vibe prompting when it includes terms that primarily describe how the image should feel rather than what discrete objects, actions, or spatial relations it should contain. Typical indicators are affective or atmospheric expressions such as «*warm, cozy, tense, nostalgic, mysterious, eerie, calm, dreamlike, or melancholic*». The core decision rule is therefore straightforward: if the prompt mainly specifies visible entities, composition, or scene structure, no vibe code is assigned; if it explicitly invokes mood, atmosphere, or emotional tone, vibe prompting is present.

After the three individual rounds have been coded, the case is coded at sequence level. At this level, the analytical object is the ordered progression of function codes across rounds. Formally, if the three round-level function codes of a case are f_{i1}, f_{i2}, f_{i3} , then the overall case type is defined as $T_i = (f_{i1}, f_{i2}, f_{i3})$, or equivalently as a concatenated notation such as F1-F2-F2. The order is analytically meaningful: F1-F2-F3 is not equivalent to F1-F3-F2. Cases are grouped into the same sequence cluster only if their ordered function sequence is identical. This is therefore an exact typological grouping rather than a distance-based clustering procedure.

The sequence coding builds directly on the round-level function codes. At sequence level, F1 represents an initial specification, F2 a referential refinement, F3 an independent re-specification, and F4 a reversion or rollback. On this basis, five sequence types are distinguished.

S1 – Iterative Refinement corresponds to the sequence $F1 \rightarrow F2 \rightarrow F2$. The user begins with an initial image specification and then refines the existing image in two consecutive edit steps. The global image concept remains stable, while later prompts optimize aspects such as framing, realism, color, placement, perspective, or mood.

S2 – Double Re-specification corresponds to $F1 \rightarrow F3 \rightarrow F3$. The sequence starts with an initial prompt, but both later rounds are formulated again as new full prompts. Instead of refining the previous output locally, the user repeatedly restates the target image in a more complete or reformulated way.

S3 – Re-specification with Final Refinement corresponds to $F1 \rightarrow F3 \rightarrow F2$. After the initial prompt, the user first reformulates the image as a new full prompt and then applies a final targeted edit to that new version. This combines a global restart with a subsequent local adjustment.

S4 – Refinement Followed by Re-specification corresponds to $F1 \rightarrow F2 \rightarrow F3$. The user first attempts a local edit of the existing image, but then abandons this refinement path and replaces it with a new full reformulation. The prompting strategy therefore shifts from incremental optimization to broader redesign.

S5 – Refinement with Reversion corresponds to $F1 \rightarrow F2 \rightarrow F4$. The sequence begins with an initial prompt and one edit prompt, but the final step returns to a previous version or corrects the prompting process itself. The trajectory is therefore no longer purely linear, because the last step is process-oriented rather than simply image-oriented.

The coding rule for the overall sequence is straightforward. First, each round is coded with its function code F1–F4. Second, the ordered sequence $T_i = (f_{i1}, f_{i2}, f_{i3})$ is constructed. Third, this sequence is matched to one of the predefined sequence types S1–S5. If a new ordered function sequence appears that is not covered by the existing typology, it is defined as a new sequence type and documented explicitly. The decisive criterion at sequence level is not the semantic content of the image, but the functional progression across rounds: if later rounds mainly adjust an existing image, the case is coded as a refinement-based sequence; if a later round restarts the image description from scratch, it is coded as a re-specification sequence; if a later round returns to an earlier state or corrects the prompting path itself, it is coded as a reversion sequence.

Overall, the codebook operationalizes each case as a three-round prompting trajectory. It combines round-level coding of function, prompt type, prompt-engineering level, and vibe prompting with sequence-level coding of the ordered function progression. This is methodologically appropriate because the dataset consists of multi-round prompt sessions in which prompting unfolds as a process of generation, refinement, reformulation, atmospheric specification, and occasional repair.

Sources

1. Liu, V., & Chilton, L. B. (2022). *Design Guidelines for Prompt Engineering Text-to-Image Generative Models*. CHI. DOI: 10.1145/3491102.3501825.
2. Xie, Y., Pan, Z., Ma, J., Jie, L., & Mei, Q. (2023). *A Prompt Log Analysis of Text-to-Image Generation Systems*. WWW '23. DOI: 10.1145/3543507.3587430.
3. Oppenlaender, J. (2024). *A taxonomy of prompt modifiers for text-to-image generation*. *Behaviour & Information Technology*. DOI: 10.1080/0144929X.2023.2286532.
4. Madaan, A., et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS. arXiv:2303.17651.

Code Book

1. Unit of analysis

One case = one person / one row / one 3-round sequence.

For each case, three units are coded separately:

- Round 1
- Round 2
- Round 3

After that, the overall sequence is coded as an additional unit.

2. Decision logic

For each round, always proceed in this order:

Step A: Assign function code

First, determine which function the prompt has in this round.

- **F1 – Generation prompt**
The prompt describes an image from scratch.
- **F2 – Edit prompt**
The prompt changes an already existing image / result.
- **F3 – Re-generation / complete reformulation**
The round appears later in the sequence, but again describes an image almost completely from scratch.
- **F4 – Meta-/repair prompt**
The prompt corrects the process, not primarily the image.
Example: “I made a mistake”, “go back to the previous image”.

Decision rule:

If the prompt clearly refers to an existing image, it is F2. If it fully describes a completely new image again, it is F3. This distinction is well supported by prompt-session research, because repeated edits and exploratory trajectories are typical there.

3. Prompt-engineering level for all prompts

Applies to F1, F2, F3 and F4. These 3 levels are your operationalization. They are informed by the literature, but not taken as a finished 3-point scale from a single source.

The assessment is based on four guiding questions:

How precise is the subject?

How clear are form / composition / perspective?

How well developed are style / modifiers / quality specifications?

Are there constraints or deliberate limitations?

L1 – low

Only the **basic** motif; **hardly any control**; very open.

Typical features:

one object or concept, no composition, hardly any style, no constraints.

Some attributes are present, but still rough and **general**.

Typical features:

object + 1–2 properties; setting only hinted at; style very general.

L2 – medium

The motif can be **clearly controlled**; several relevant **specifications** are present.

Typical features:

clear motif description, recognizable setting, initial perspective or style control.

L3 – high

The prompt clearly controls image effect, composition, and style.

Typical features:

precise spatial cues, consistent style, several detail modifiers, good controllability.

Very precise, controlled, constraint-based full prompt.

Typical features:

targeted relations, **high specific** visual logic, strong limitations, **limited** room for interpretation.

Rationale:

Liu and Chilton show that subject and style in particular are key control elements;

Oppenlaender shows that practitioners systematically work with prompt modifiers. From this, a quality assessment can be cleanly derived through specificity, style control, and constraints.

4. Codebook for the overall sequence

The final analytical level does not examine prompts in isolation, but classifies the entire prompt trajectory of a case. This step is necessary because the dataset consists of multi-round prompting sessions rather than independent single prompts. In such sessions, the analytical object is not only what users prompt, but how they move from one prompt to the next. The sequence level therefore captures the user's overall prompting strategy across rounds. This perspective is methodologically justified because text-to-image prompting typically unfolds as an iterative session process rather than as a one-shot act. Users begin with an initial specification and then either refine, reformulate, or repair the request in subsequent rounds. The sequence code therefore captures the procedural logic of prompting beyond the individual round.

4.1 Analytical principle: case type as an ordered function sequence

At the sequence level, each case is represented as an **ordered function sequence**. Let a case i consist of X_i rounds, where $X_i \in \{2,3,4\}$. In the present dataset, the empirical material consists of three-round cases, so operationally $X_i = 3$, but the formulation remains generalizable.

Let

$$f_{i1} = \text{function of Round 1}, f_{i2} = \text{function of Round 2}, \dots, f_{iX_i} = \text{function of Round } X_i$$

Then the **case type** is defined as:

$$T_i = (f_{i1}, f_{i2}, \dots, f_{iX_i})$$

Equivalently, as a concatenated notation:

$$T_i = R1_Function_i \oplus R2_Function_i \oplus \dots \oplus RX_Function_i$$

where $\oplus$ denotes the ordered concatenation of function codes. Thus, a case may be represented for example as:

$$T_i = F1 - F2 - F2$$

This formulation is important because the order of functions is analytically meaningful:

$$F1 - F2 - F3 \neq F1 - F3 - F2$$

Hence, the sequence is treated as a **structured progression**, not as an unordered set of codes.

4.2 Sequence clustering rule

All cases with the same ordered function sequence are assigned to the same sequence cluster. Formally, for a given sequence type t ,

$$C_t = \{i \mid T_i = t\}$$

The total number of sequence clusters is therefore the number of unique ordered function sequences in the dataset:

$$K = |\{T_i \mid i = 1, \dots, n\}|$$

This is not a distance-based clustering procedure, but an **exact typological grouping**. Cases are grouped together only if their functional sequence is identical. This is methodologically appropriate because the variables at issue are nominal function codes rather than metric features. A procedure such as k-means would therefore be inappropriate, whereas exact sequence-based grouping preserves both category identity and temporal order.

4.3 Working definition of the function codes

The sequence coding builds directly on the function codes already assigned at round level:

- **F1 – Initial specification**
The prompt describes an image from scratch and establishes the first full visual target.
- **F2 – Referential refinement**
The prompt refers to an already generated image and introduces a targeted modification.
- **F3 – Independent re-specification**
The prompt appears later in the sequence but again formulates the image as a new full prompt rather than as a local revision.
- **F4 – Reversion / rollback**
The prompt does not primarily refine the image itself, but reorients the process by returning to an earlier version or correcting the previous prompting path.

These function definitions are the basis for the higher-level sequence types below.

4.4 Rationale for sequence-level typology

The sequence level is introduced because prompt sessions are not sufficiently described by round-wise coding alone. Two cases may both contain one generation prompt and two later prompts, yet differ fundamentally in strategy: one may consist of steady local refinement, another of repeated full rewrites, and another of a rollback to an earlier state. The sequence type therefore captures the **mode of progression** through which the user approaches the image.

This makes the coding scheme sensitive to differences between:

- stable incremental optimization,
- repeated global reformulation,
- mixed strategies,
- and recursive repair behavior.

In methodological terms, the sequence code increases interpretive precision because it describes not only prompt content, but also the **interaction logic** of the prompting session.

4.5 Sequence types

Based on the ordered function sequences observed in the dataset, five sequence types are distinguished:

S1 – Iterative Refinement

Sequence: $F1 \rightarrow F2 \rightarrow F2$

Definition:

The user begins with an initial image specification and then refines the existing image in two consecutive edit steps.

Explanation:

The global image concept remains stable across the sequence. Subsequent prompts do not replace the scene, but optimize particular aspects such as framing, realism, color, object placement, perspective, or mood.

Justification:

This is the clearest form of progressive refinement. Both later rounds are referential and incremental. The user works on the same emerging image and improves it step by step. This type therefore represents a stable feedback-and-adjustment trajectory.

S2 – Double Re-specification

Sequence: $F1 \rightarrow F3 \rightarrow F3$

Definition:

The sequence starts with an initial prompt, but both later rounds are formulated again as new full prompts.

Explanation:

Instead of refining the previous output locally, the user repeatedly restates the target image in a more complete or reformulated manner. The prompting strategy shifts from editing to re-describing.

Justification:

This type indicates that the user does not treat the existing image as a stable reference point. Rather than fine-tuning a prior output, the user repeatedly reconstructs the desired image globally. It therefore represents a strongly exploratory or re-anchoring strategy.

S3 – Re-specification with Final Refinement

Sequence: $F1 \rightarrow F3 \rightarrow F2$

Definition:

After the initial prompt, the user first reformulates the image as a new full prompt and then applies a final targeted edit to that new version.

Explanation:

The sequence combines a global restart with a subsequent local adjustment. The user first redefines the image concept, then fine-tunes the revised target.

Justification:

This pattern reflects a two-step strategy: first, conceptual correction at the global level; second, controlled polishing at the local level. It is therefore neither a purely incremental nor a purely rewrite-based process, but a hybrid form.

S4 – Refinement Followed by Re-specification

Sequence: $F1 \rightarrow F2 \rightarrow F3$

Definition:

The user first attempts a local edit of the existing image, but then abandons this refinement path and replaces it with a new full reformulation.

Explanation:

The sequence begins as incremental optimization but later shifts toward a more fundamental redesign of the target image.

Justification:

This type is analytically important because it captures a change in prompting mode. The user initially assumes that local changes may be sufficient, but later concludes that a broader reformulation is needed. It therefore marks a move from local correction to global reorientation.

S5 – Refinement with Reversion

Sequence: $F1 \rightarrow F2 \rightarrow F4$

Definition:

The sequence begins with an initial prompt and one edit prompt, but the final step explicitly returns to a previous version or corrects the prompting process itself.

Explanation:

The prompting path is no longer linear. A later state is not simply refined further, but is partially rejected in favor of an earlier state or earlier image version.

Justification:

This type captures recursive repair behavior. The final prompt is process-oriented rather than image-oriented in the narrow sense. It signals dissatisfaction not only with image details, but with the current revision trajectory itself.

4.6 Coding rule for the overall sequence

After rounds 1, 2, and 3 have been coded individually, the overall sequence is coded by concatenating the round-level function codes in their original order.

Procedure:

1. Code each round with its function code $F1$ – $F4$.
2. Construct the ordered sequence:

$$T_i = (f_{i1}, f_{i2}, f_{i3})$$

or, in general form,

$$T_i = (f_{i1}, \dots, f_{iX_i})$$

3. Match the resulting sequence to one of the predefined sequence types $S1$ – $S5$.
4. If a new function sequence occurs that is not covered by the existing typology, define it as a new sequence type and document it explicitly.

4.7 Decision rule

The decisive criterion at sequence level is not the semantic content of the image, but the **functional progression across rounds**.

- If later rounds mainly adjust an existing image, code as a refinement-based sequence.
- If a later round restarts the image description from scratch, code as a re-specification sequence.
- If a later round returns to an earlier state or corrects the prompting path itself, code as a reversion sequence.

Thus, the overall sequence code answers the question:

What kind of prompting trajectory does this case represent?

4.8 Summary

The overall-sequence codebook operationalizes each case as an ordered function sequence and groups cases into exact sequence types. This approach is methodologically appropriate because the dataset consists of multi-round prompt sessions in which prompting unfolds as a process of generation, refinement, reformulation, and occasional repair. The sequence level therefore complements round-level coding by capturing the user's higher-order prompting strategy.